

Федеральное государственное автономное образовательное учреждение высшего образования «**Национальный исследовательский университет ИТМО**»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №4
по дисциплине «**Основы программной инженерии**»

Вариант: 386132

Преподаватель:
Кулинич Ярослав Вадимович

Выполнил: Некрутенко Максим
Группа: P3206

Оглавление

1. Задание.....	3
2. Выполнение.....	4
1. Исходный код разработанных MBean.....	4
2. JConsole.....	8
3. VisualVM.....	11
4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе.	13
Вывод.....	19

1. Задание

Лабораторная работа #4

Вариант 386132

Внимание! У разных вариантов разный текст задания!

1. Для своей программы из лабораторной работы #3 по дисциплине "Веб-программирование" реализовать:

- MBeap, считающий общее число установленных пользователем точек, а также число точек, попадающих в область. В случае, если пользователь совершил 3 "промаха" подряд, разработанный MBeap должен отправлять оповещение об этом событии.
- MBeap, определяющий средний интервал между кликами пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBeap-классов, разработанных в ходе выполнения задания 1.
- Определить версию Java Language Specification, реализуемую данной средой исполнения.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBeap-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя потока, потребляющего наибольший процент времени CPU.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе.

По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

2. Выполнение

1. Исходный код разработанных MBean

- AreaStatsMBean

```
1 package org.example.mbean;
2
3
4 public interface AreaStatsMBean { 1 usage 1 implementation new *
5     int getTotalPoints(); no usages 1 implementation new *
6     int getHitPoints(); no usages 1 implementation new *
7     int getMissPoints(); no usages 1 implementation new *
8     int getConsecutiveMisses(); no usages 1 implementation new *
9     void resetStats(); no usages 1 implementation new *
10 }
```

- AreaStats

```
package org.example.mbean;

import javax.management.*;
import java.util.concurrent.atomic.AtomicInteger;

public class AreaStats extends NotificationBroadcasterSupport implements AreaStatsMBean {
    private AtomicInteger totalPoints = new AtomicInteger(0);
    private AtomicInteger hitPoints = new AtomicInteger(0);
    private AtomicInteger missPoints = new AtomicInteger(0);
    private AtomicInteger consecutiveMisses = new AtomicInteger(0);

    private long sequenceNumber = 0;

    @Override
    public int getTotalPoints() {
        return totalPoints.get();
    }

    @Override
    public int getHitPoints() {
        return hitPoints.get();
    }

    @Override
    public int getMissPoints() {
        return missPoints.get();
    }

    @Override
    public int getConsecutiveMisses() {
        return consecutiveMisses.get();
    }
}
```

```

public void addPoint (boolean isHit) {
    totalPoints.incrementAndGet();

    if (isHit) {
        hitPoints.incrementAndGet();
        consecutiveMisses.set(0);
    } else{
        missPoints.incrementAndGet();
        int misses = consecutiveMisses.incrementAndGet();
        if (misses == 3) {
            sendTreeMissesNotification();
        }
    }
}

private void sendTreeMissesNotification() {
    Notification notification = new Notification(
        "area.stats.three.misses",
        this,
        ++sequenceNumber,
        System.currentTimeMillis(),
        "Warning: 3 consecutive misses detected!"
    );

    sendNotification(notification);
}

@Override
public void resetStats() {
    totalPoints.set(0);
    hitPoints.set(0);
    missPoints.set(0);
    consecutiveMisses.set(0);
}
}

```

- ClickTimingMBean

```

package org.example.mbean;

public interface ClickTimingMBean {
    double getAverageInterval();
    long getTotalClick();
    void resetTiming();
}

```

- ClickTiming

```
package org.example.mbean;

import java.util.concurrent.atomic.AtomicLong;

public class ClickTiming implements ClickTimingMBean {
    private AtomicLong totalClick = new AtomicLong(0);
    private AtomicLong lastClickTime = new AtomicLong(0);
    private AtomicLong totalInterval = new AtomicLong(0);

    @Override
    public double getAverageInterval() {
        long click = totalClick.get();
        if (click <= 1) return 0.0;
        return (double) totalInterval.get() / (click - 1);
    }

    @Override
    public long getTotalClick() {
        return totalClick.get();
    }

    public void recordClick() {
        long now = System.currentTimeMillis();
        long prev = lastClickTime.getAndSet(now);

        if (prev != 0) {
            long interval = now - prev;
            totalInterval.addAndGet(interval);
        }
        totalClick.incrementAndGet();
    }

    @Override
    public void resetTiming() {
        totalClick.set(0);
        lastClickTime.set(0);
        totalInterval.set(0);
    }
}
```

- JmxRegistrar

```
package org.example.mbean;

import javax.faces.bean.ApplicationScoped;
import javax.faces.bean.ManagedBean;
import javax.annotation.PostConstruct;
import javax.management.MBeanServer;
import javax.management.ObjectName;
import java.lang.management.ManagementFactory;

@ManagedBean(name= "jmxRegistrar", eager = true)
@ApplicationScoped
public class JmxRegistrar {
    private AreaStats areaStats;
    private ClickTiming clickTiming;

    @PostConstruct
    public void init() {
        try {

```

```

MBeanServer mBeanServer = ManagementFactory.getPlatformMBeanServer();

areaStats = new AreaStats();
ObjectName areaStatName = new ObjectName("org.example.mbean:type=AreaStats");
mBeanServer.registerMBean(areaStats, areaStatName);

clickTiming = new ClickTiming();
ObjectName clickTimingName = new ObjectName("org.example.mbean:type=ClickTiming");
mBeanServer.registerMBean(clickTiming, clickTimingName);

System.out.println("MBeans зарегистрированы");

} catch (Exception e) {
    System.err.println("Ошибка регистрации MBeans:");
    e.printStackTrace();
}
}

public AreaStats getAreaStats() {
    return areaStats;
}

public ClickTiming getClickTiming() {
    return clickTiming;
}
}

```

2. JConsole

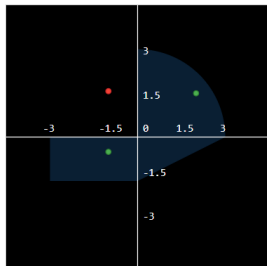
Показания MBean-классов, разработанных в ходе выполнения задания 1:

• Метаданные Mbean'ов:

Координата X:
Координата Y:
Радиус R:

• Result Miss

X	Y	R	Результат	Время
-1.0	1.5659999	3.0	Miss	01.06.2026 18:45:05
-1.0	-0.50400007	3.0	Hit	01.06.2026 18:45:04
2.0	1.4940001	3.0	Hit	01.06.2026 18:45:01

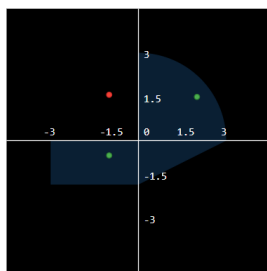


[Вернуться на главную страницу](#)

Координата X:
Координата Y:
Радиус R:

• Result Miss

X	Y	R	Результат	Время
-1.0	1.5659999	3.0	Miss	01.06.2026 18:45:05
-1.0	-0.50400007	3.0	Hit	01.06.2026 18:45:04
2.0	1.4940001	3.0	Hit	01.06.2026 18:45:01



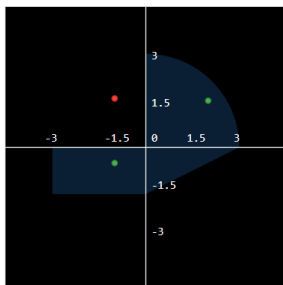
[Вернуться на главную страницу](#)

• Показания Mbean'ов:

Координата X:
Координата Y:
Радиус R:

• Result Miss

X	Y	R	Результат	Время
-1.0	1.5659999	3.0	Miss	01.06.2026 18:45:05
-1.0	-0.50400007	3.0	Hit	01.06.2026 18:45:04
2.0	1.4940001	3.0	Hit	01.06.2026 18:45:01

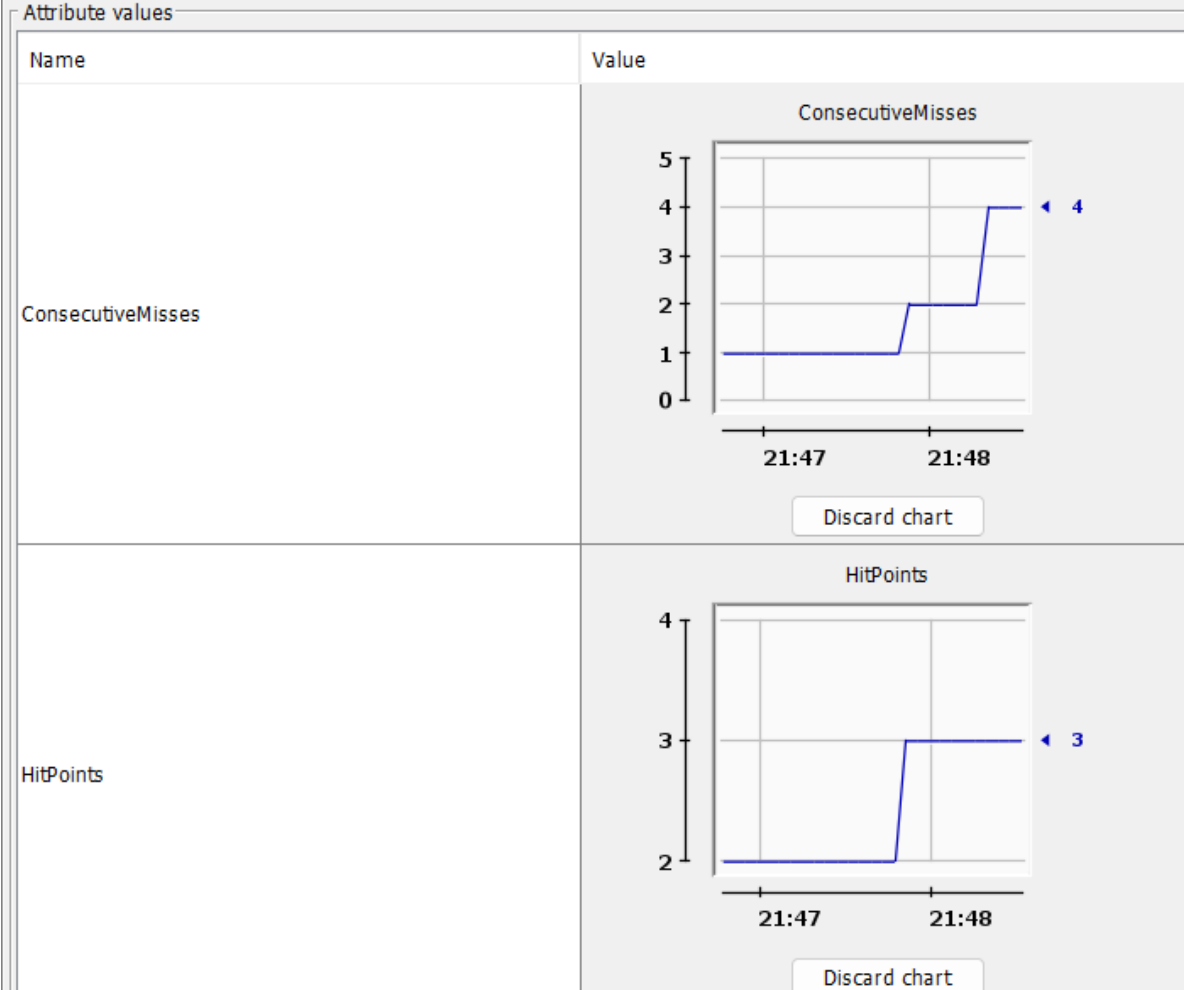


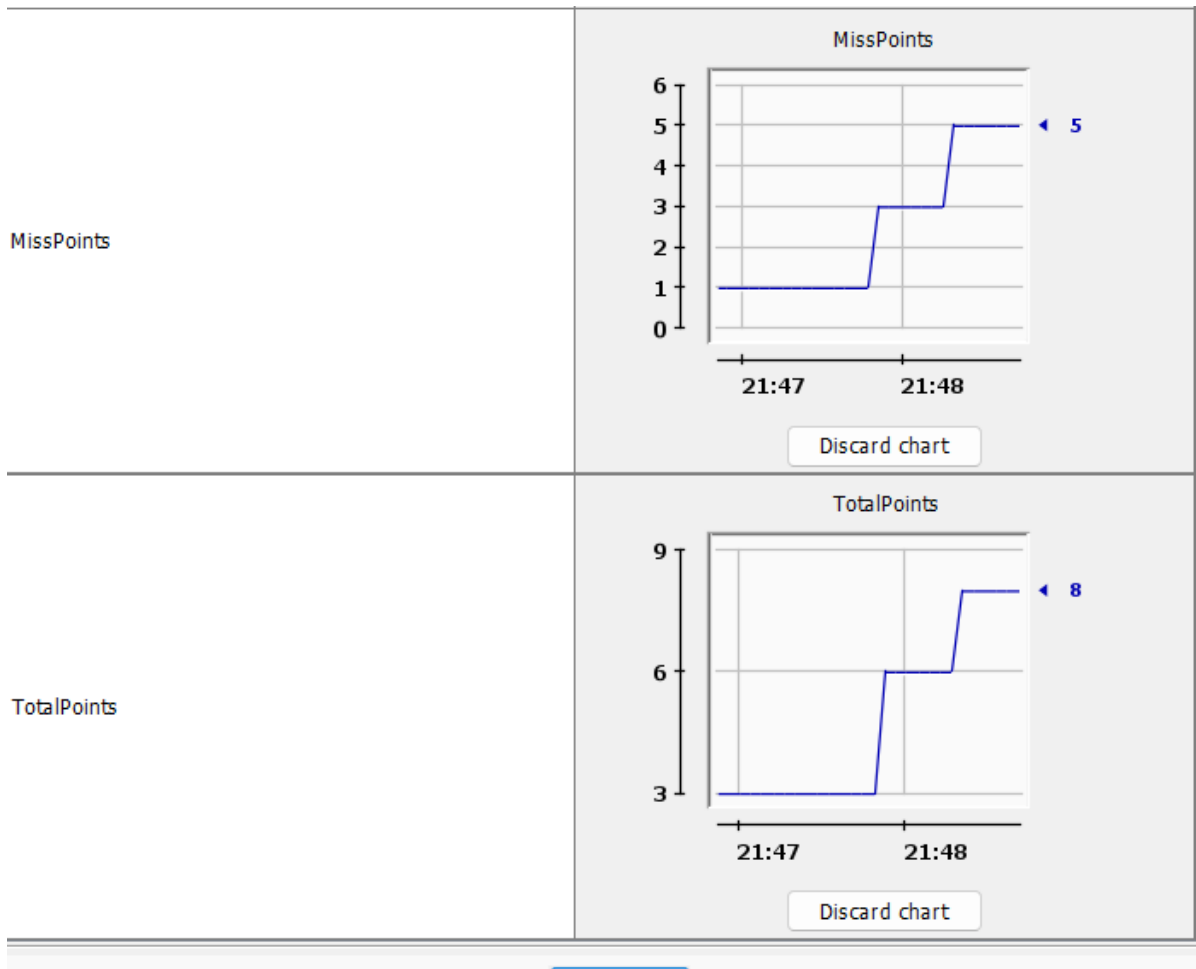
[Вернуться на главную страницу](#)

- **VM Summary** и время, прошедшее с момента запуска виртуальной машины:

VM Summary	
понедельник, 1 июня 2026 г., 21:46:26 Москва, стандартное время	
Connection name: admin@servicejmxremote=http://localhost:9990	Uptime: 2 minutes
Virtual Machine: OpenDK 64-Bit Server VM version 17.0.12-7-1	Process CPU time: 1 minute
Vendor: OpenDK BSD Porting Team	JIT compiler: HotSpot 64-Bit Tiered Compilers
Name: 33598@helios.cs.ifmo.ru	Total compile time: 1 minute
Live threads: 107	Current classes loaded: 26 566
Peak: 190	Total classes loaded: 26 577
Daemon threads: 30	Total classes unloaded: 11
Total threads started: 207	
Current heap size: 168 193 kbytes	Committed memory: 382 976 kbytes
Maximum heap size: 1 048 576 kbytes	Pending finalization: 0 objects
Garbage collector: Name = 'G1 Young Generation', Collections = 37, Total time spent = 0.156 seconds	
Garbage collector: Name = 'G1 Old Generation', Collections = 0, Total time spent = 0.000 seconds	
Operating System: FreeBSD 14.4-STABLE	Total physical memory: 134 161 832 kbytes
Architecture: amd64	Free physical memory: 88 768 900 kbytes
Number of processors: 16	Total swap space: 67 108 864 kbytes
Committed virtual memory: 4 260 920 kbytes	Free swap space: 67 108 864 kbytes
VM arguments: -D[Standalone] -Xms4m -Xmx512m -XXMaxMetaspaceSize=96M -XXMaxMetaspaceSize=256m -Djava.net.preferIPv4Stack=true -Djboss.modules.system.pkgs=org.jboss.byteman -Djava.awt.headless=true --add-exports=java.desktop/sun.awt=ALL-UNNAMED --add-exports=java.naming/com.sun.jndi.map=ALL-UNNAMED --add-exports=java.naming/com.sun.jndi.url.map=ALL-UNNAMED --add-exports=java.naming/com.sun.jndi.url.map=ALL-UNNAMED --add-exports=java.base/java.lang.invoke=ALL-UNNAMED --add-exports=java.base/java.lang.reflect=ALL-UNNAMED --add-exports=java.base/java.io=ALL-UNNAMED --add-exports=java.base/java.security=ALL-UNNAMED --add-exports=java.base/java.util.concurrent=ALL-UNNAMED --add-exports=java.management.javax.management=ALL-UNNAMED --add-exports=java.naming.javax.naming=ALL-UNNAMED -Djava.security.manager=allow -Dorg.jboss.boot.log.file=/home/studs/s466853/wildfly-26.1.3.Final/standalone/log/server.log -Dlogging.configuration=file:/home/studs/s466853/wildfly-26.1.3.Final/standalone/configuration/logging.properties -XXMaxHeapSize=1G -XXMaxMetaspaceSize=1024m	
Class path: /home/studs/s466853/wildfly-26.1.3.Final/jboss-modules.jar	
Library path: /usr/java/packages/lib/amd64:/usr/lib:/usr/local/lib	
Boot class path: Unavailable	

- **Графики:**





• Уведомления:

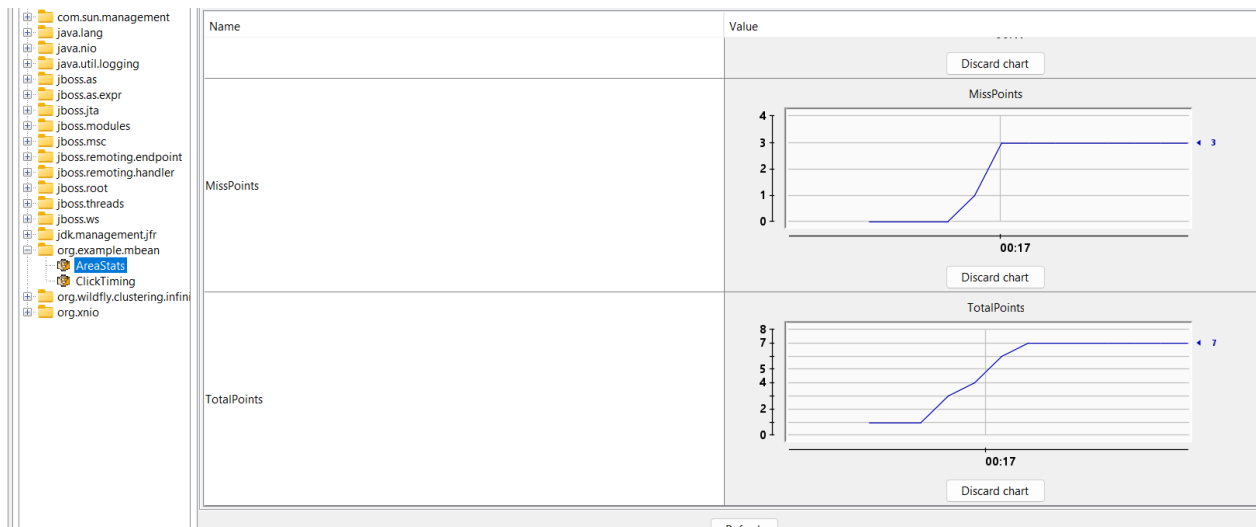
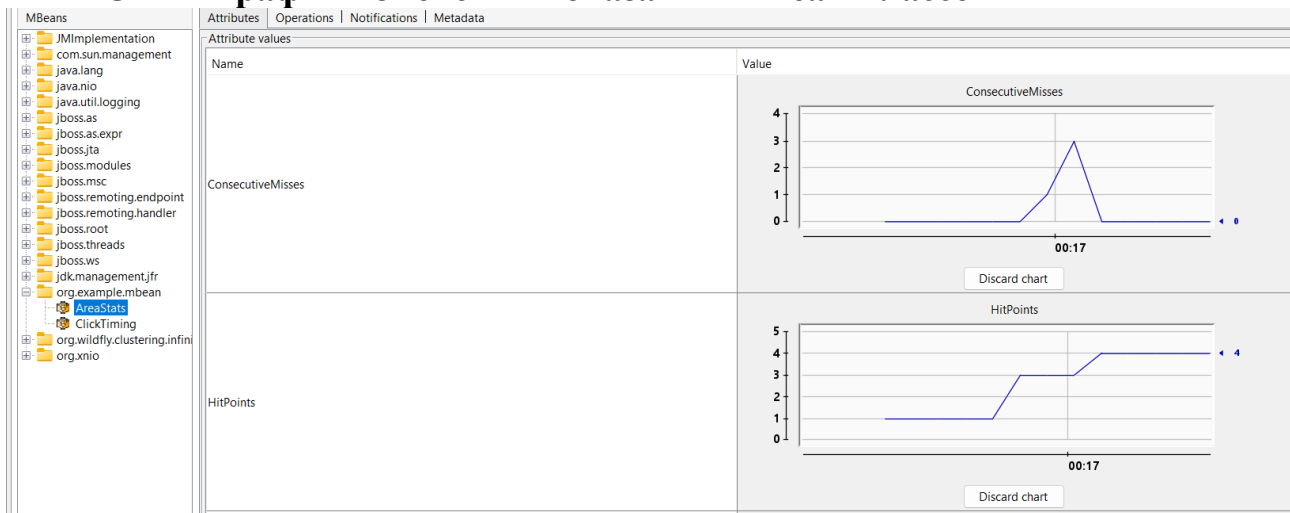
Connection Window Help							
Overview	Memory	Threads	Classes	VM Summary	MBeans	WildFly CLI	
Notification buffer							
Timestamp	Type	UserData	SeqNum	Message	Event	Source	
21:55:43:468	area.stats.three.misses		7	Warning: 3 consecutive misses detected	java.management.Notification{sourc...	org.example.mbean:type=AreaStats	
21:55:34:784	area.stats.three.misses		6	Warning: 3 consecutive misses detected	java.management.Notification{sourc...	org.example.mbean:type=AreaStats	
21:55:29:425	area.stats.three.misses		5	Warning: 3 consecutive misses detected	java.management.Notification{sourc...	org.example.mbean:type=AreaStats	
21:55:19:759	area.stats.three.misses		4	Warning: 3 consecutive misses detected	java.management.Notification{sourc...	org.example.mbean:type=AreaStats	
21:55:10:441	area.stats.three.misses		3	Warning: 3 consecutive misses detected	java.management.Notification{sourc...	org.example.mbean:type=AreaStats	

- **Выводы по результатам мониторинга:**

В ходе мониторинга с использованием утилиты JConsole я узнал время работы JVMs, успешно разработал и зарегистрировал Mbean. В случае трех последовательных промахов MBean `AreaStats` отправляет уведомления, которые были получены с помощью JConsole. Таким образом, было зарегистрировано и протестировано получение уведомлений от Mbean.

3. VisualVM

- **Снять график изменения показаний MBean-классов:**



• JVM Overview:

jboss-modules.jar (pid 58969)

Overview

PID: 58969
Host: localhost
Main class: jboss-modules.jar
Arguments: -mp /home/studs/s466853/wildfly-26.1.3.Final/modules org.jboss.as.standalone -Djboss.home.dir=/home/studs/s466853/wildfly-26.1.3.Final -Djboss.server.base.dir=/home/studs/s466853/wildfly-26.1.3.Final/standalone -b 0.0.0.0 -Djboss.server.port=9990

JVM: OpenJDK 64-Bit Server VM (17.0.12+7-1, mixed mode, sharing)
Java: version 17.0.12 2024-07-16, vendor OpenJDK BSD Porting Team
Java Home: /usr/local/openjdk17
JVM Flags: <none>

Heap dump on OOME: disabled

Thread Dumps: 0
Heap Dumps: 0
Profiler Snapshots: 0
JFR Snapshots: 0

JVM arguments:

- D[Standalone]
- Xms64m
- Xmx512m
- XX:MetaspaceSize=96M
- XX:MaxMetaspaceSize=256m
- Djava.net.preferIPv4Stack=true
- Djboss.modules.system.pkgs=org.jboss.byteman
- Djava.awt.headless=true
- add-exports=java.desktop/sun.awt=ALL-UNNAMED
- add-exports=java.naming/com.sun.jndi.idap=ALL-UNNAMED
- add-exports=java.naming/com.sun.jndi.url.idap=ALL-UNNAMED
- add-exports=java.naming/com.sun.jndi.url.idaps=ALL-UNNAMED
- add-opens=java.base/java.lang=ALL-UNNAMED
- add-opens=java.base/java.lang.invoke=ALL-UNNAMED
- add-opens=java.base/java.lang.reflect=ALL-UNNAMED
- add-opens=java.base/java.io=ALL-UNNAMED
- add-opens=java.base/java.security=ALL-UNNAMED
- add-opens=java.base/java.util=ALL-UNNAMED
- add-opens=java.base/java.util.concurrent=ALL-UNNAMED
- add-opens=java.management/javax.management=ALL-UNNAMED
- add-opens=java.naming/javax.naming=ALL-UNNAMED
- Djava.security.manager=allow
- Dorg.jboss.boot.log.file=/home/studs/s466853/wildfly-26.1.3.Final/standalone/log/server.log

• Мониторинг:

jboss-modules.jar (pid 54592)

Threads

Live threads: 108
Daemon threads: 29

Timeline

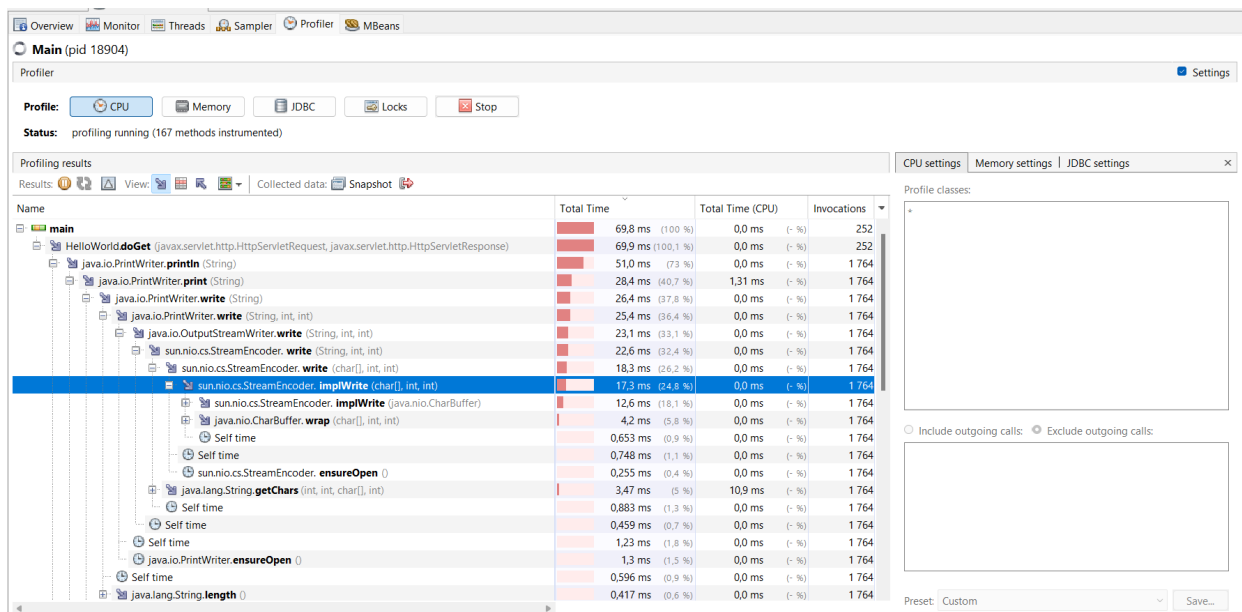
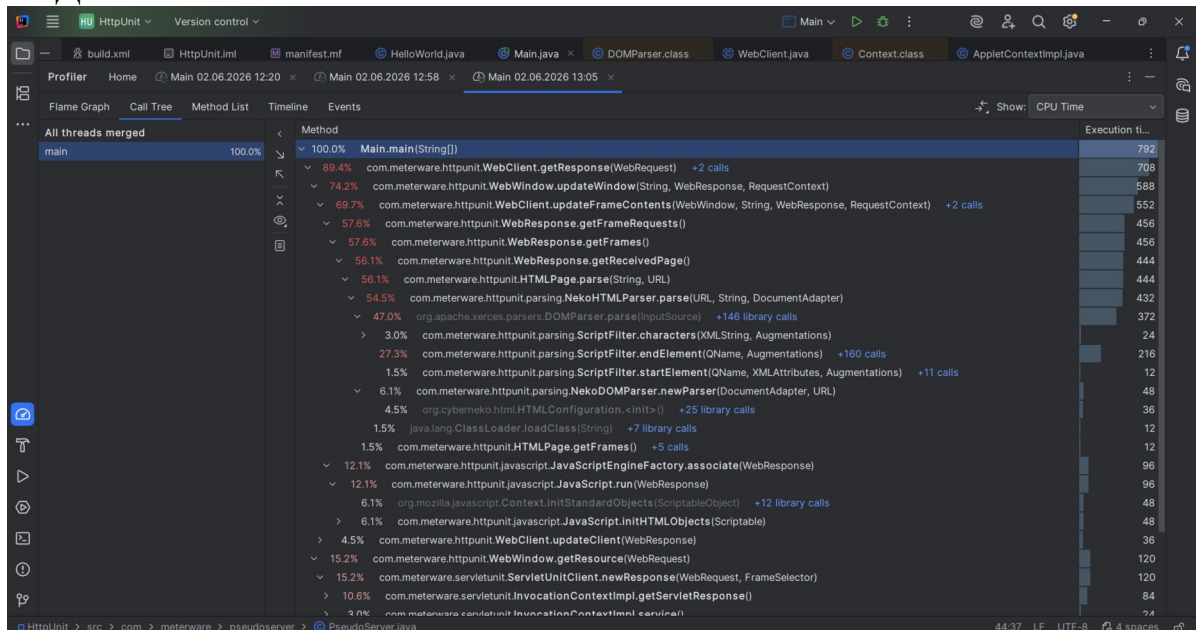
Show: Live Threads | Timeline: 🔍

Name	16:50:55	16:51:00	16:51:05	16:51:10	16:51:15	16:51:20	16:51:25	Running	Total
default Accept								285 427 ms (100 %)	285 427 ms
default I/O-1								285 427 ms (100 %)	285 427 ms
default I/O-10								285 427 ms (100 %)	285 427 ms
default I/O-11								285 427 ms (100 %)	285 427 ms
default I/O-12								285 427 ms (100 %)	285 427 ms
default I/O-13								285 427 ms (100 %)	285 427 ms
default I/O-14								285 427 ms (100 %)	285 427 ms
default I/O-15								285 427 ms (100 %)	285 427 ms
default I/O-16								285 427 ms (100 %)	285 427 ms
default I/O-17								285 427 ms (100 %)	285 427 ms
default I/O-18								285 427 ms (100 %)	285 427 ms
default I/O-19								285 427 ms (100 %)	285 427 ms
default I/O-2								285 427 ms (100 %)	285 427 ms
default I/O-20								285 427 ms (100 %)	285 427 ms
default I/O-21								285 427 ms (100 %)	285 427 ms
default I/O-22								285 427 ms (100 %)	285 427 ms
default I/O-23								285 427 ms (100 %)	285 427 ms
default I/O-24								285 427 ms (100 %)	285 427 ms
default I/O-25								285 427 ms (100 %)	285 427 ms
default I/O-26								285 427 ms (100 %)	285 427 ms
default I/O-27								285 427 ms (100 %)	285 427 ms

Running Sleeping Wait Park Monitor

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе.

До



```

public class Main {
}

/**
 * @param args the command line arguments
 */
public static void main(String[] args) {
    try {
        HttpUnitOptions.setExceptionsThrownOnScriptError(false);
        ServletRunner sr = new ServletRunner();
        sr.registerServlet( resourceName: "myServlet", HelloWorld.class.getName());
        ServletUnitClient sc = sr.newClient();
        int number = 1;
        WebRequest request = new GetMethodWebRequest( urlString: "http://test.meterware.com/myServlet");
        while (true) {
            WebResponse response = sc.getResponse(request);
            System.out.println("Count: " + number++ + response);
            java.lang.Thread.sleep( millis: 200);
        }
    } catch (InterruptedException ex) {
        Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
    } catch (MalformedURLException ex) {
        Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
    } catch (IOException ex) {
        Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
    } catch (SAXException ex) {
        Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
    }
}
}

```

```

/**
 *
 * @author gs145266
 */
public class HelloWorld extends HttpServlet {

    @Override no usages
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        PrintWriter out = response.getWriter();
        response.setContentType("text/html");
        out.println("<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.0 \" +
            \"Transitional//EN\">\n" +
            "<HTML>\n" +
            "<HEAD><TITLE>Hello World</TITLE></HEAD>\n" +
            "<BODY>\n" +
            "<H1>Hello World</H1>\n");
        out.println("<script language=\"JavaScript\" type=\"text/javascript\">");
        out.println("<!--");
        out.println("document.wr('Hello Document')");
        out.println("//-->");
        out.println("</script>");
        out.println("</BODY></HTML>");
    }
}

```

ПОСЛЕ

Method	Execution tim...
100.0% Main.main(String[])	7,601
96.0% com.meterware.httpunit.WebClient.getResponse(WebRequest) +5 calls	7,297
81.8% com.meterware.httpunit.WebWindow.updateWindow(String, WebResponse, RequestContext) +21 calls	6,215
76.6% com.meterware.httpunit.WebResponse.getReceivedPage() +24 calls	5,825
66.5% org.apache.xerces.parsers.DOMParser.parse(InputSource) +404 library calls	5,058
< 1% com.meterware.httpunit.parsing.ScriptFilter.endElement(QName, Augmentations)	15
< 1% com.meterware.httpunit.parsing.ScriptFilter.startElement(QName, XMLAttributes, Augmentations)	256
< 1% com.meterware.httpunit.parsing.ScriptFilter.endElement(QName, Augmentations)	30
9.3% com.meterware.httpunit.parsing.NekoDOMParser.newParser(DocumentAdapter, URL)	707
< 1% java.lang.ClassLoader.loadClass(String) +31 library calls	15
3.6% com.meterware.httpunit.HTMLPage.getFrames()	270
13.8% com.meterware.httpunit.WebWindow.getResource(WebRequest)	1,052
12.8% com.meterware.servletunit.ServletUnitClient.newResponse(WebRequest, FrameSelector)	976
< 1% com.meterware.httpunit.HttpUnitUtils.isJavaScriptURL(String)	46
< 1% com.meterware.httpunit.WebClient.tellListeners(WebResponse)	15
< 1% java.lang.ClassLoader.loadClass(String) +4 library calls	15
2.4% java.io.PrintStream.println(String) +22 library calls	183
< 1% java.lang.String.valueOf(Object)	30
< 1% java.lang.invoke.Invokers\$Holder.linkToTargetMethod(int, Object, Object) +4 library calls	30

Overview

Monitor

Threads

Sampler

Profiler

MBeans

Main (pid 15972)

Profiler

Profile:

CPU

Memory

JDBC

Locks

Stop

Status:

profiling running (166 methods instrumented)

Profiling results

Results: View: Collected data: Snapshot

Name	Total Time	Total Time (CPU)	Invocations
main	8 472 ms (100.0 %)	0,0 ms (- %)	38 048
HelloWorld.doGet (javax.servlet.http.HttpServletRequest, javax.servlet.http.HttpServletResponse)	8 478 ms (100,1 %)	0,0 ms (- %)	38 048
java.io.PrintWriter.println (String)	7 366 ms (86,9 %)	0,0 ms (- %)	266 331
java.io.PrintWriter.print (String)	3 760 ms (44,4 %)	0,0 ms (- %)	266 331
java.io.PrintWriter.write (String)	3 618 ms (42,7 %)	0,0 ms (- %)	266 331
java.io.PrintWriter.write (String, int, int)	3 397 ms (40,1 %)	0,0 ms (- %)	266 331
java.io.OutputStreamWriter.write (String, int, int)	3 259 ms (38,5 %)	0,0 ms (- %)	266 331
sun.nio.cs.StreamEncoder.write (String, int, int)	3 203 ms (37,8 %)	0,0 ms (- %)	266 331
sun.nio.cs.StreamEncoder.write (char[], int, int)	2 460 ms (29 %)	0,0 ms (- %)	266 331
sun.nio.cs.StreamEncoder.implWrite (char[], int, int)	2 318 ms (27,4 %)	0,0 ms (- %)	266 331
sun.nio.cs.StreamEncoder.implWrite (java.nio.CharBuffer)	1 790 ms (21,1 %)	0,0 ms (- %)	266 331
java.nio.charset.CharsetEncoder.encode (java.nio.CharBuffer, java.nio.ByteBuffer, boolean)	1 326 ms (15,7 %)	0,0 ms (- %)	266 331
sun.nio.cs.ISO_8859_1\$Encoder.encodeLoop (java.nio.CharBuffer, java.nio.ByteBuffer)	1 116 ms (13,2 %)	0,0 ms (- %)	266 331
sun.nio.cs.ISO_8859_1\$Encoder.encodeArrayLoop (java.nio.CharBuffer, java.nio.ByteBuffer)	891 ms (10,5 %)	0,0 ms (- %)	266 331
Self time	299 ms (3,5 %)	21,2 ms (- %)	266 331
sun.nio.cs.ISO_8859_1\$Encoder.encodeISOArray (char[], int, byte[], int, int)	286 ms (3,4 %)	0,0 ms (- %)	266 331
java.nio.CharBuffer.position (int)	105 ms (1,2 %)	0,0 ms (- %)	266 331
java.nio.ByteBuffer.position (int)	85,1 ms (1 %)	0,0 ms (- %)	266 331
java.nio.CharBuffer.arrayOffset ()	29,9 ms (0,4 %)	0,0 ms (- %)	266 331
java.nio.ByteBuffer.arrayOffset ()	29,5 ms (0,3 %)	0,0 ms (- %)	266 331
java.nio.ByteBuffer.array ()	29,0 ms (0,3 %)	0,0 ms (- %)	266 331
java.nio.CharBuffer.array ()	26,4 ms (0,3 %)	0,0 ms (- %)	266 331


```

public class Main {
    public Main() { 2 usages
    }

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        try {
            com.meterware.httpunit.HttpUnitOptions.setScriptingEnabled(false);
            HttpUnitOptions.setExceptionsThrownOnScriptError(false);
            ServletRunner sr = new ServletRunner();
            sr.registerServlet( resourceName: "myServlet", HelloWorld.class.getName());
            ServletUnitClient sc = sr.newClient();
            int number = 1;
            WebRequest request = new GetMethodWebRequest( urlString: "http://test.meterware.com/myServlet");
            while (true) {
                WebResponse response = sc.getResponse(request);
                System.out.println("Count: " + number++ + response);
                // java.lang.Thread.sleep(200);
            }
        } catch (MalformedURLException ex) {
            Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
        } catch (IOException ex) {
            Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
        } catch (SAXException ex) {
            Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
        }
    }
}

```

```

public class HelloWorld extends HttpServlet {

    @Override no usages
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Объединяем весь HTML в один StringBuilder, чтобы убрать 7 последовательных вызовов println
        StringBuilder html = new StringBuilder();
        html.append("<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.0 Transitional//EN\">\n")
            .append("<HTML>\n")
            .append("<HEAD><TITLE>Hello World</TITLE></HEAD>\n")
            .append("<BODY>\n")
            .append("<H1>Hello World</H1>\n")
            .append("<script language=\"JavaScript\" type=\"text/javascript\">\n")
            .append("<!--\n")
            .append("document.wr('Hello Document')\n")
            .append("//-->\n")
            .append("</script>\n")
            .append("</BODY></HTML>");

        out.print(html.toString());
        out.flush();
    }
}

```


Описание выявленной проблемы:

В ходе анализа производительности исследуемого Java-приложения в среде разработки IntelliJ IDEA и внешней утилите VisualVM были обнаружены критические архитектурные дефекты, приводившие к сильному падению производительности и неэффективной утилизации вычислительных мощностей.

Проблема №1:

Встроенный профилировщик IDE (вкладки Flame Graph и Call Tree)

зафиксировал, что главный поток приложения `main` непрерывно загружен.

Около **40% всего процессорного времени** уходило на выполнение методов библиотеки обработки JavaScript — `ScriptFilter.endElement` и движка Rhino (`org.mozilla.javascript.Context.evaluateString`). Библиотека `HttpUnit` при обработке каждого HTTP-ответа от сервера тратила ресурсы CPU на компиляцию и интерпретацию скриптов, которые никак не проверялись тестовым сценарием клиента.

Проблема №2:

Дерево вызовов утилиты VisualVM показало, что метод сервлета

`HelloWorld.doGet()` тратит **74.4% времени** исключительно на выполнение системных вызовов ввода-вывода через метод

`java.io.PrintWriter.println(String)`. Из-за пошаговой, построчной отправки HTML-разметки (8 последовательных вызовов `println`), поток блокировался на время отрисовки и перекодирования каждого фрагмента текста на экране консоли/сервера.

Описание путей устранения выявленной проблемы:

1) Поскольку выполнение JavaScript-кода на целевой странице сервлета никак не задействовано в логике интеграционного теста, поддержка сценариев была отключена аппаратно. Это позволило полностью исключить вызовы тяжелого интерпретатора Rhino. Из бесконечного цикла также удалена искусственная задержка `Thread.sleep(200)`, маскировавшая реальную скорость работы алгоритма.

2) Множественные вызовы `out.println` заменены на буферизацию всей страницы в оперативной памяти с помощью эффективного класса `StringBuilder`. Запись в выходной сетевой поток теперь осуществляется однократно.

Подробное описание алгоритма действий, который позволил выявить и локализовать проблему (скриншоты представлены вначале задания 4):

Шаг 1. Исходный код приложения был запущен в IntelliJ IDEA в режиме сбора диагностических данных с помощью инструмента Run with Profiler (Java Flight Recorder)

Шаг 2. Проведена визуальная оценка распределения нагрузки во вкладке **Flame Graph**. Обнаружен доминирующий по ширине сине-серый блок вызовов `o.m.j.Context.evaluateString`, находящийся на самой вершине пирамиды вызовов потока `main`, что явно указало на проблему с обработкой JavaScript.

Шаг 3. Детальный структурный анализ проведен во вкладке **Call Tree** в IDE. Инструмент подтвердил затраты 27.3% процессорного времени на метод `ScriptFilter.endElement` и 12.1% времени на инициализацию `JavaScript.run`

Шаг 4. Для исследования серверной части было выполнено подключение внешней утилиты **VisualVM** к запущенному процессу. В настройках вкладки Profiler -> CPU были полностью деактивированы стандартные фильтры (очищено поле `Exclude outgoing calls`, в `Profile classes` установлена маска *)

Шаг 5. Полученное дерево вызовов VisualVM локализовало проблему ввода-вывода: метод `HelloWorld.doGet` тратил 74.4% времени внутри метода блокирующей печати `java.io.PrintWriter.println`.

Шаг 6. На основе данных профилирования в исходный код внесены исправления: в клиентском классе `Main.java` с помощью директивы `setScriptingEnabled(false)` аппаратно отключен встроенный интерпретатор JavaScript, а в серверном классе `HelloWorld.java` множественные вызовы `out.println` заменены на эффективную буферизацию HTML-страницы в оперативной памяти через класс `StringBuilder`. Из бесконечного цикла клиента также удалена задержка `Thread.sleep(200)` для оценки пиковой пропускной способности приложения.

Шаг 7. Оптимизированное приложение перезапущено в среде разработки IntelliJ IDEA. Вкладка **Monitor** внешней утилиты VisualVM и вкладка **Call Tree** профилировщика IDEA подтвердили успешность внесенных изменений: затраты на обработку элементов скриптов упали до уровня **меньше 1%** (15 мс), а тяжелые методы выполнения JS и блокирующей печати `PrintWriter.println` полностью покинули топ ресурсоемких операций программы.

Вывод

В ходе выполнения лабораторной работы были освоены практические навыки мониторинга, профилирования и оптимизации производительности распределенных Java-приложений с использованием диагностического инструментария среды разработки IntelliJ IDEA и утилиты VisualVM и JConsole. Были получены навыки выявления и устранения вычислительных дефектов классов, а также верификации результатов оптимизации по графикам распределения ресурсов JVM и временным шкалам активности потоков.